



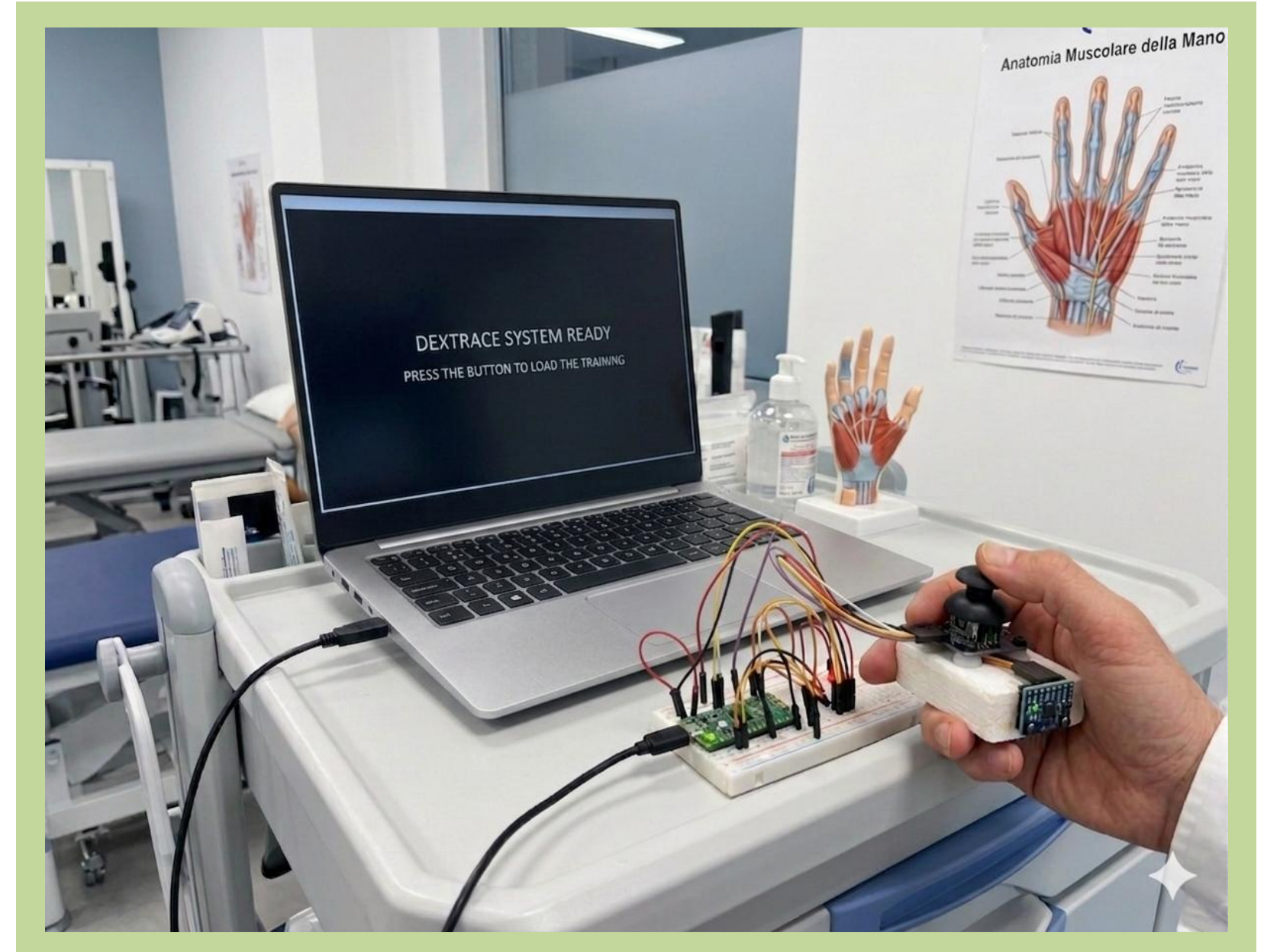
Rehabilitation for hand fine motor coordination and dexterity

The project

DexTrace is an integrated hardware-software system designed to **train** and **rehabilitate hand fine motor coordination** and **dexterity**.

By combining a **Raspberry Pi Pico** controller with a **Python-based GUI**, the system evaluates user performance across three progressive difficulty levels, monitoring **spatial precision** and **tremor intensity** in **real-time**.

The key concept is to recover from injuries in a playful way.



Clinical Evaluation Protocol

- Level 1 (*Reaching*): Path efficiency and target acquisition assessment;
- Level 2 (*Tracking*): Smooth circular target following;
- Level 3 (*Chasing*): Non-linear dynamics to simulate complex scenarios.

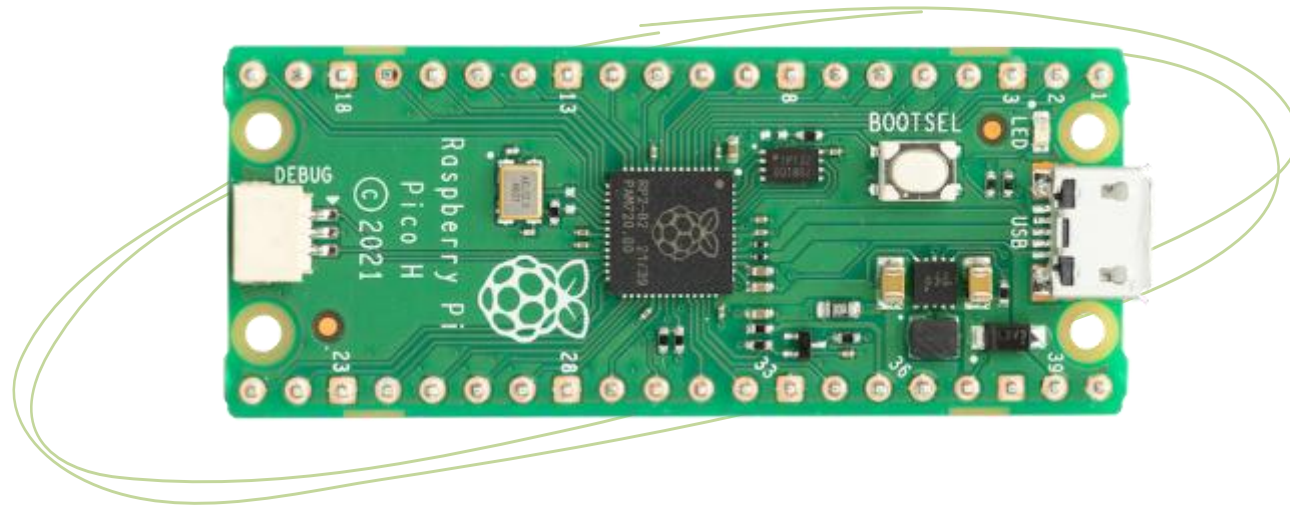
Dex-Metrics

Metric	Description	Formula	Level
Tremor intensity	Calculated based on the variance of 3-axis acceleration from the MPU6050	$Var(Mag_{accel})$	All
Dex-Efficiency	Ratio between real and optimal path + tremor penalty	$Path_{eff} + (Tremor_{avg} \cdot 0.5)$	1
Dex-Accuracy	RMSE weighted by tremor intensity	$RMSE + (Tremor_{avg} \cdot 10)$	2 & 3

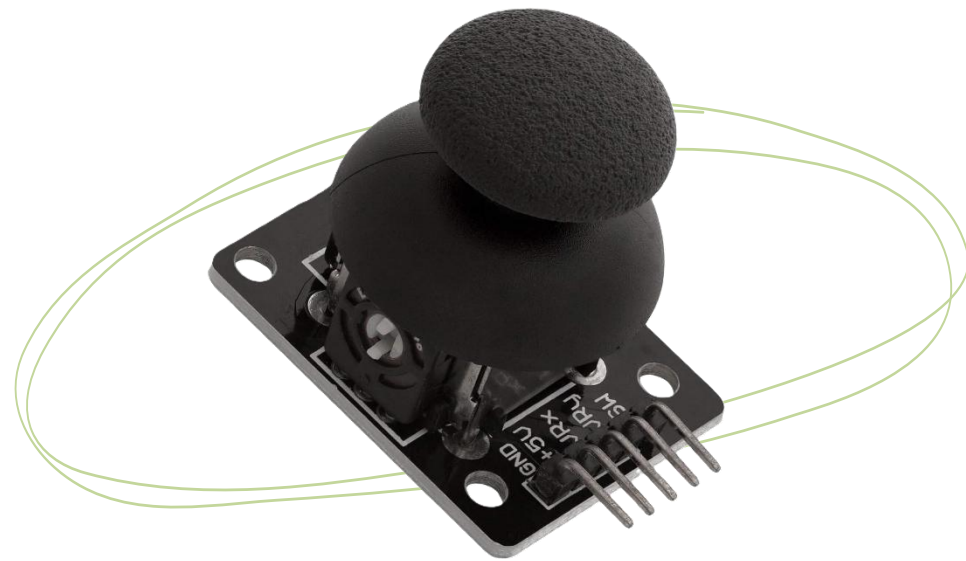
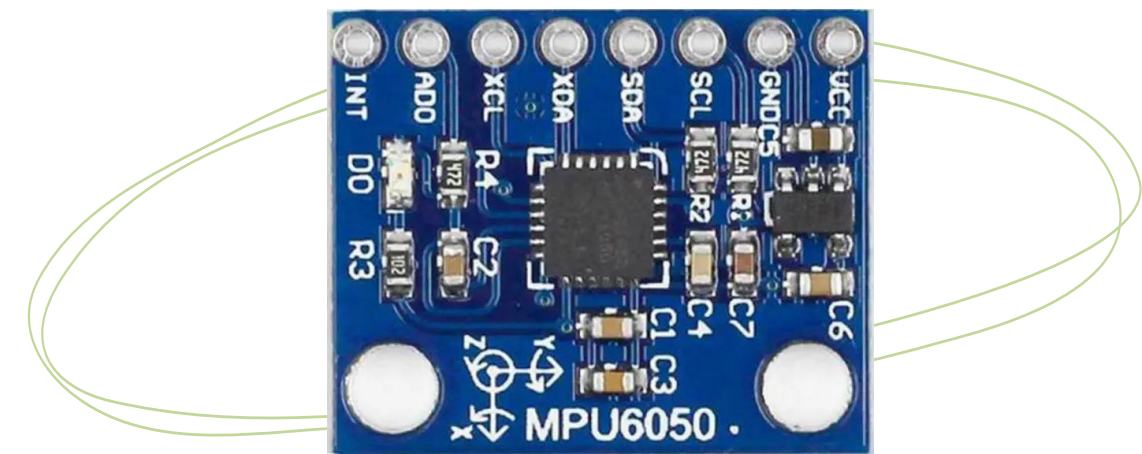
System Requirements

Hardware

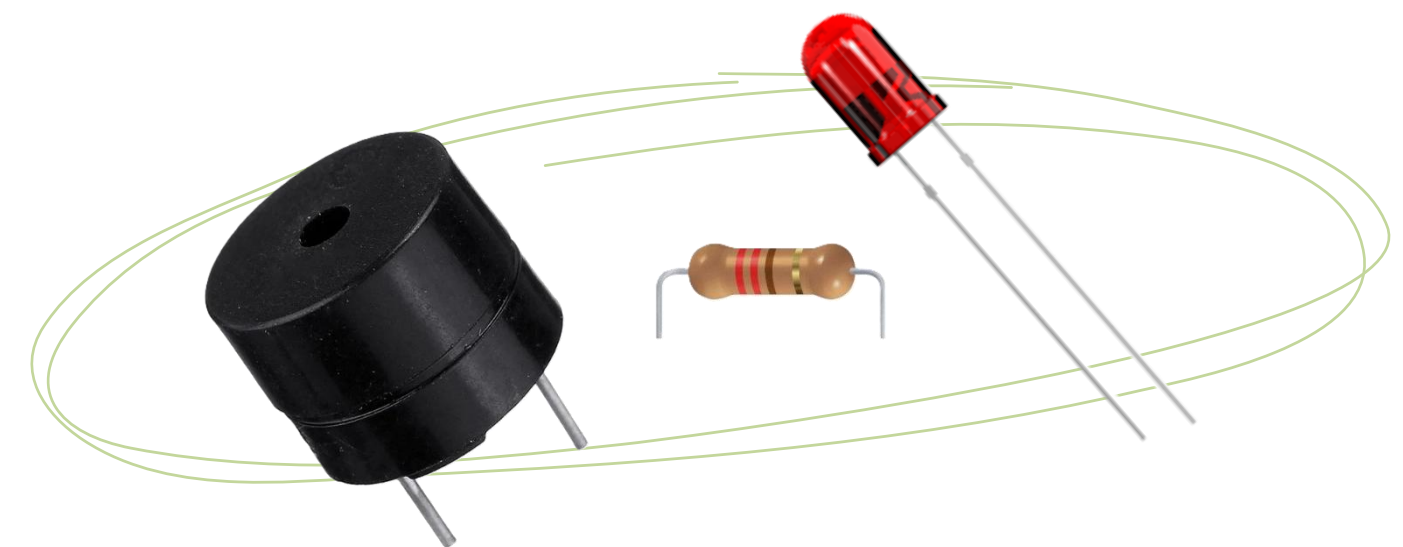
Raspberry PI Pico



IMU sensor



Analog Joystick (2-axis with integrated button)



Active buzzer and Led

System Requirements

Software



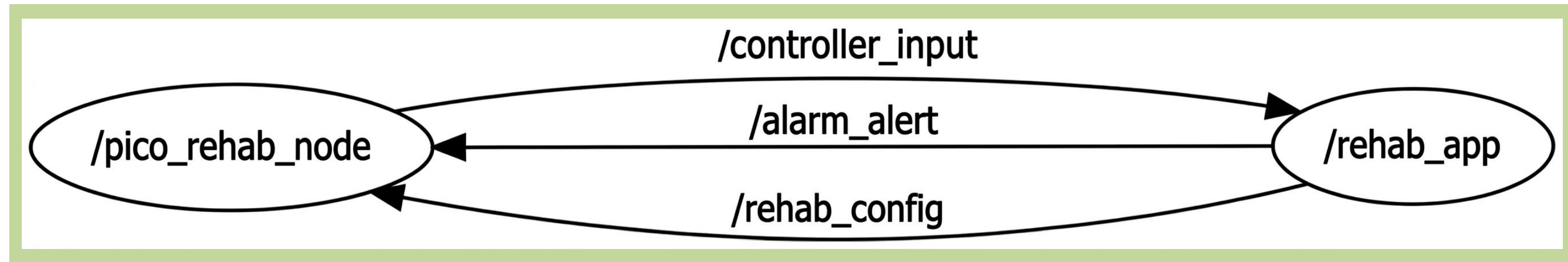
Wiring Table

PICO PIN	Component PIN
38	Joystick GND, IMU GND
36	Joystick VCC, IMU VCC
31 – GP26	Joystick VRx
32 – GP27	Joystick VRy
27 – GP21	Joystick SW
7 – GP5	IMU SCL
6 – GP4	IMU SDA
20 – GP15	Buzzer
21 – GP16	Led

} 12-bit ADC

} I2C

Communication ROS2 + uROS



Nodes:

- `pico_rehab_node`
- `rehab_app`



Raspberry PI Pico



PC

Topics:

- `rehab_config`
- `controller_input`
- `alarm_alert`



Level configuration



Joystick and IMU



Acoustic alarm

Messages

/rehab_config

geometry_msgs/msg/Twist

linear.x = Level gain
linear.y = Eval threshold

Pico is a subscriber

/controller_input

geometry_msgs/msg/Twist

linear.x = Joystick VRx*gain
linear.y = Joystick VRy*gain
angular.x = Tremor intensity
angular.z = System state

Pico is a publisher

/alarm_alert

std_msgs/msg/Bool

data = target_alarm

Pico is a subscriber

Standard message types with a custom field mapping (only for Twist messages) to optimize the bandwidth

Data Structures

```
struct SystemState {  
    bool active;  
    bool target_alarm;  
    float current_tremor_intensity;  
    DeviceConfig_t config;  
};
```

This approach guarantee:

- temporal coherence;
- producer-consumer decoupling.

SystemState keeps the current state active or not of the system, the target_alarm from the topic, the current_tremor_intensity from the IMU and the config received from the rehap_app node, defined as:

```
typedef struct {  
    float gain;  
    float tremor_threshold;  
} DeviceConfig_t;
```

The configuration parameters are level-dependant. Its access is protected by a **Mutex**.

```
typedef struct {  
    float x;  
    float y;  
    float tremor;  
    bool active;  
} SensorPacket_t;
```

SensorPacket_t is the current snapshot of the sensors ready to be sent. Each packet is filled in the **xSensorQueue**, with **xQueueOverwrite** (dim = 1) to keep only the last new fresh packet.

Task Scheduling & priorities

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Buzzer	Async	Event driven	4	1	Manages real-time acoustic feedback for target chase with minimum latency
Tremor	10 ms	100 Hz	4	1	Samples MPU6050 raw data via I2C and computes tremor intensity using variance analysis
Input	10 ms	100 Hz	3	1	Handles ADC joystick readings, dynamic gain scaling, and button software debouncing
ROS	30+10 ms	25 Hz	2	0	Executes the micro-ROS executor, manages serial transport, and implements the safety watchdog
Heartbeat	1000 ms	1 Hz	1	0	Provides visual system health status and session state via the onboard LED blinking frequency.

Task Buzzer

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Buzzer	Async	Event driven	4	1	Manages real-time acoustic feedback for target chase with minimum latency

- Event-Driven:** The task is purely asynchronous. By using Task Notifications, it remains in a power-saving blocked state with zero CPU overhead, waking up only when signaled by the ROS 2 callback (`/alarm_alert`).
- Real-Time Analysis:** The 160ms cycle (80ms ON/OFF) is optimized for ergonomic clarity. The logic ensures that the buzzer completes its current semi-period before silencing, avoiding electrical glitches.
- Jitter Analysis:** Isolation on Core 1 with maximum priority prevents interference from heavy ROS communication, providing a stable and coherent stimulus for neuro-motor rehabilitation.

Task Tremor

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Tremor	10 ms	100 Hz	4	1	Samples MPU6050 raw data via I2C and computes tremor intensity using variance analysis

Real-Time Analysis: This task must be deterministic. If the accelerometer sampling (via I2C) is delayed, the variance calculation over 20 samples (sliding window) becomes physically meaningless.

Global Resources: Writes to the `global_state` (Mutex-protected) the `current_tremor_intensity` computed.

Local Feedback: Immediate visual alert via LED if intensity exceeds threshold, no ROS communication jitter.

Task Input

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Input	10 ms	100 Hz	3	1	Handles ADC joystick readings, dynamic gain scaling, and button software debouncing

Real-Time Analysis: It manages the ADC (Analog-to-Digital Converter). There is a software filtering option to prevent electrical noise from the ADC from moving the cursor when the joystick is idle. It has the same T as the **Task Tremor** and based on the **Rate Monotonic Scheduling** they should have the same priorities, but the decision is to give more importance to the signal precision.

Debouncing: The button is handled in software with a `vTaskDelay(50)`. It didn't use interrupts to avoid electrical glitches that could crash the Levels FSM (Finite State Machine).

Task ROS

TASK	T	F	PRIORITY	CORE	DESCRIPTION
ROS	30+10 ms	25 Hz	2	0	Executes the micro-ROS executor, manages serial transport, and implements the safety watchdog

Period:

From 30 to 40 ms due to

```
rclcpp_executor_spin_some(&executor, RCL_MS_TO_NS(10));  
vTaskDelay(pdMS_TO_TICKS(30));
```

Real-Time Analysis:

This is the most demanding and unpredictable task. The micro-ROS stack is complex and manages serial buffers, so this task is assigned to Core 0, protecting hardware tasks (Core 1) from potential slowdowns caused by USB/Serial communication.

Watchdog:

Implementation of a Fail-Safe design. If the PC stops sending data the Pico enters a safe state.

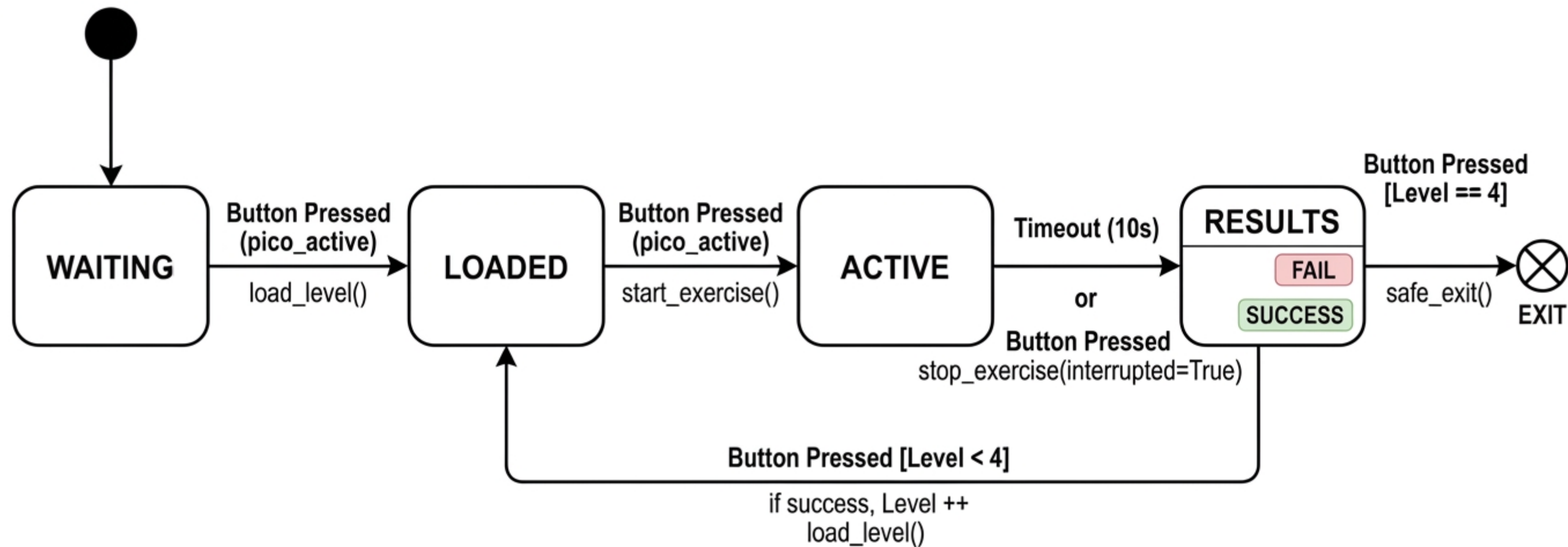
Task Heartbeat

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Heartbeat	1000 ms	1 Hz	1	0	Provides visual system health status and session state via the onboard LED blinking frequency.

Real-Time Analysis: This is a "best-effort" task. It runs only when the CPU has nothing better to do. Changing the blink rate based on the active state is an excellent way to perform visual debugging without external tools.

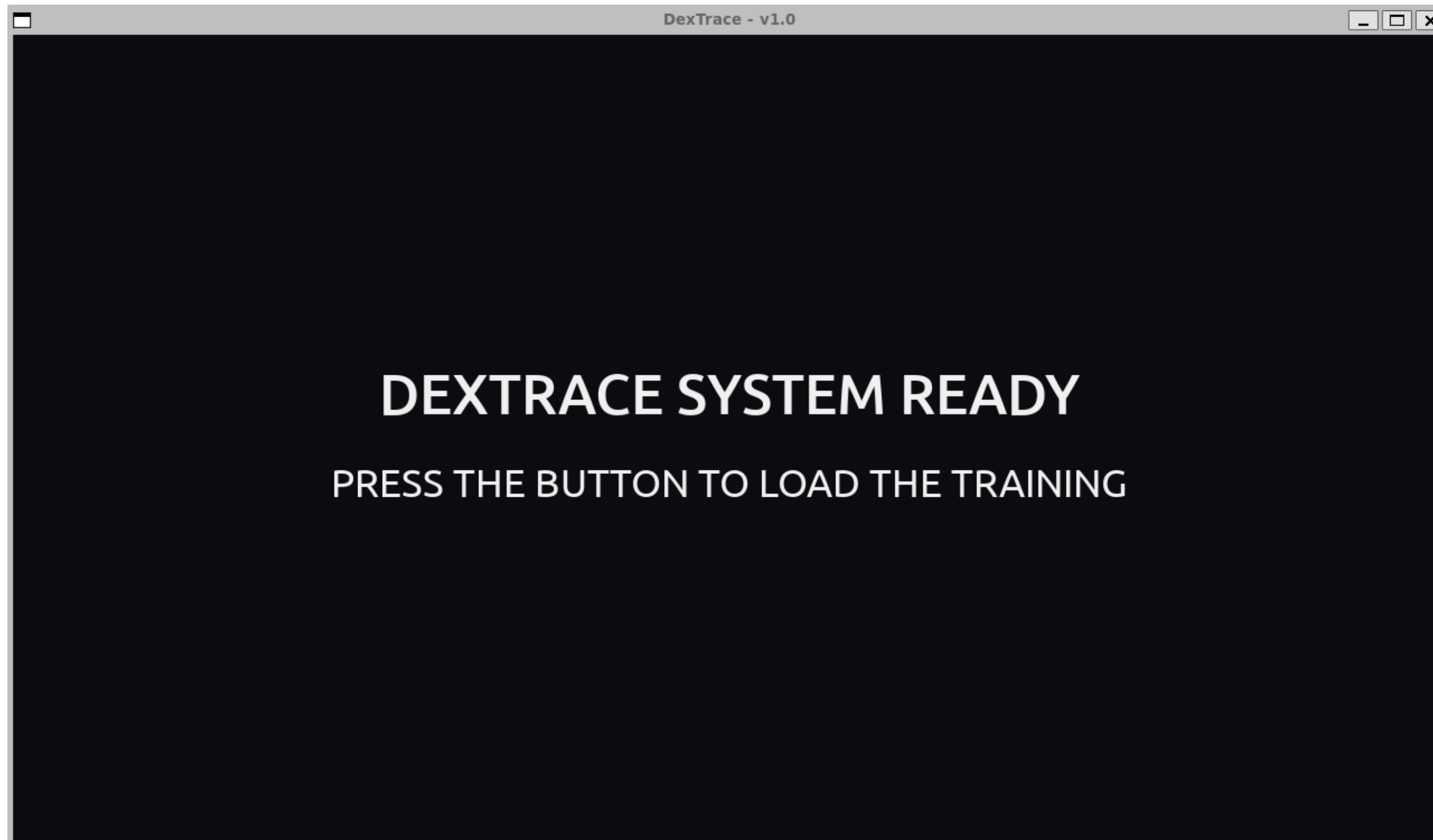
GUI - FSM

DEXTRACE



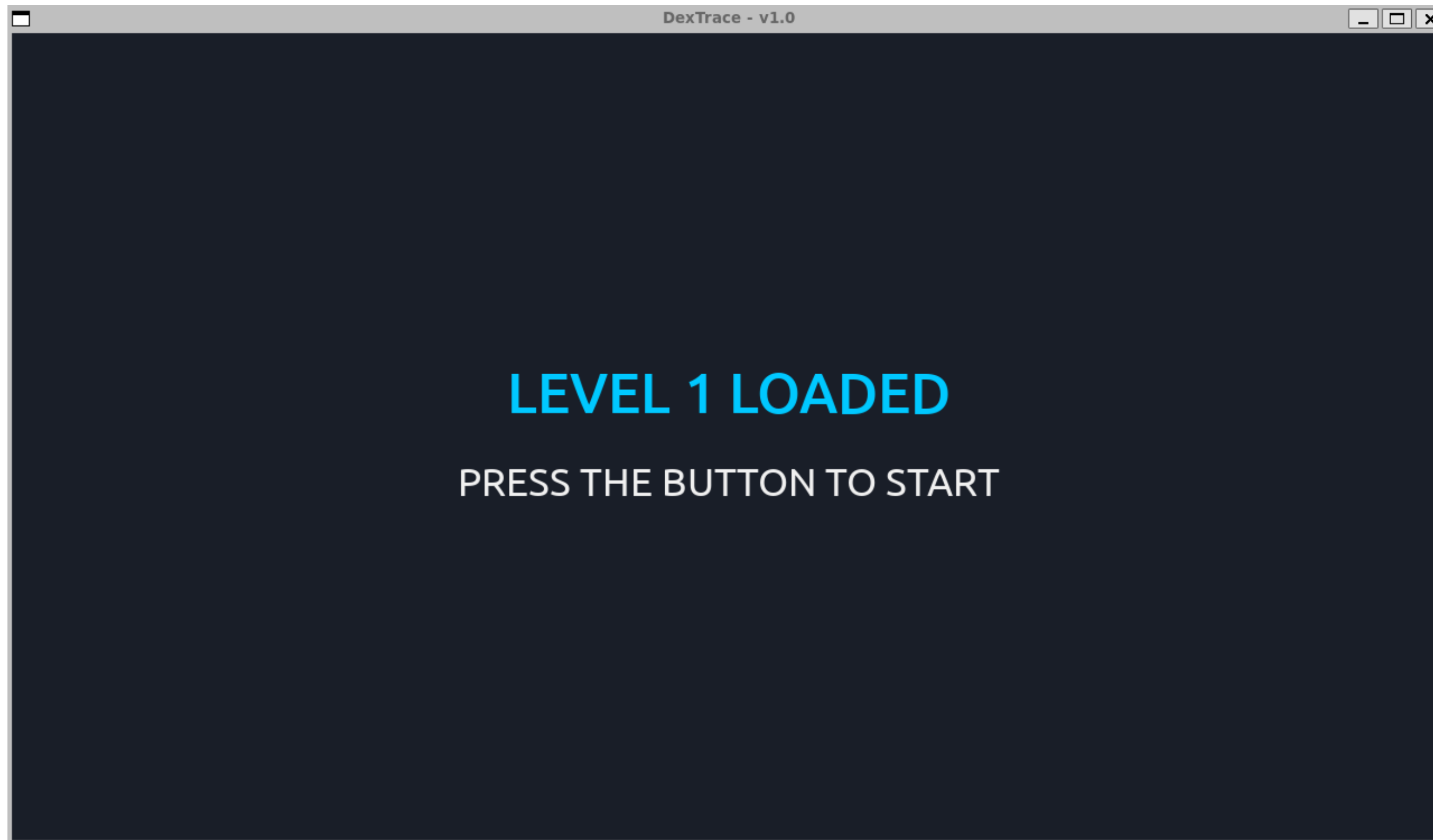
DEXTRACE LEVELS: 📍 1: REACHING 📍 2: TRACKING 📍 3: CHASING 📍 4: COMPLETE

Homepage



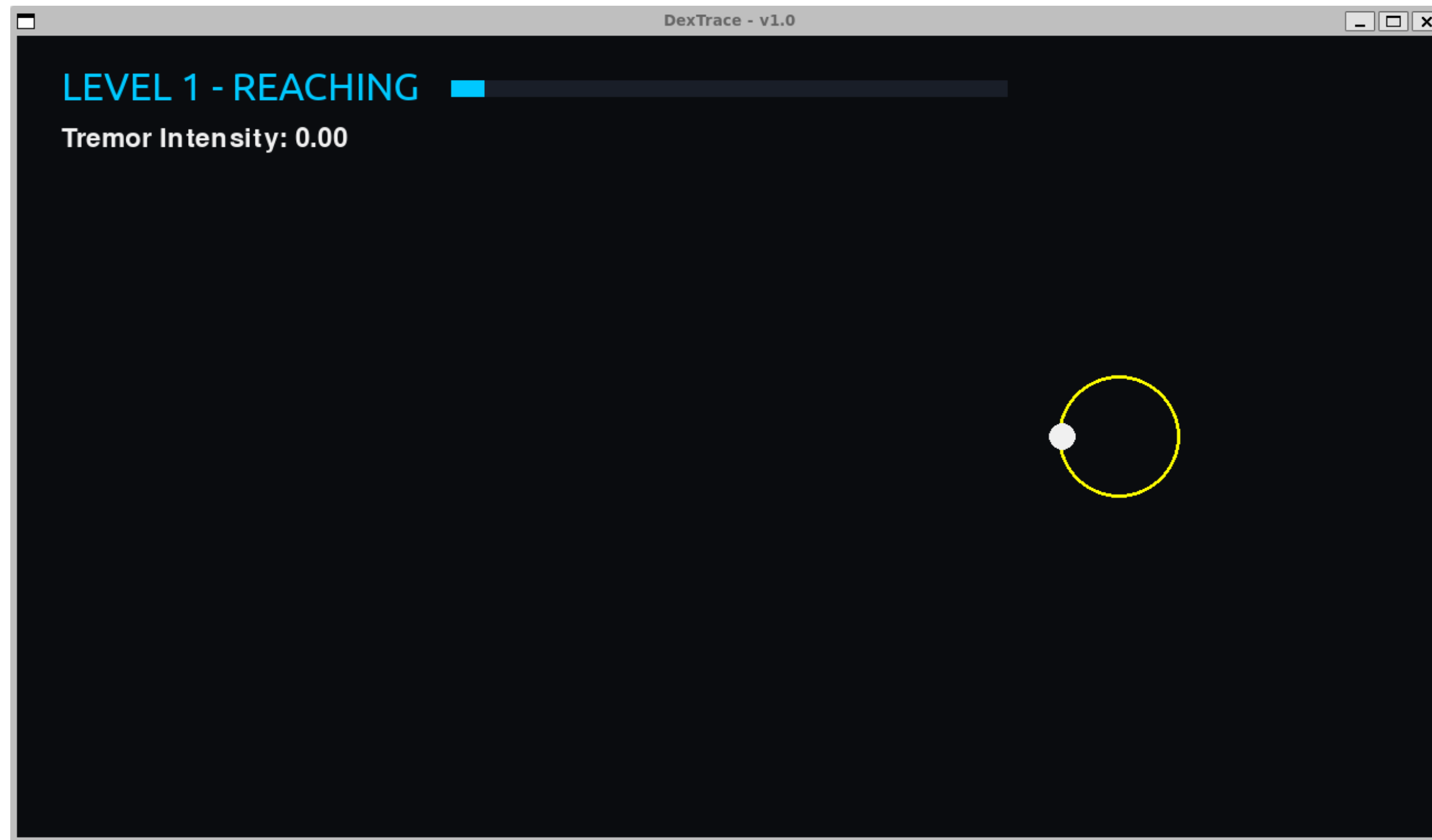
Library: pygame

Load Level



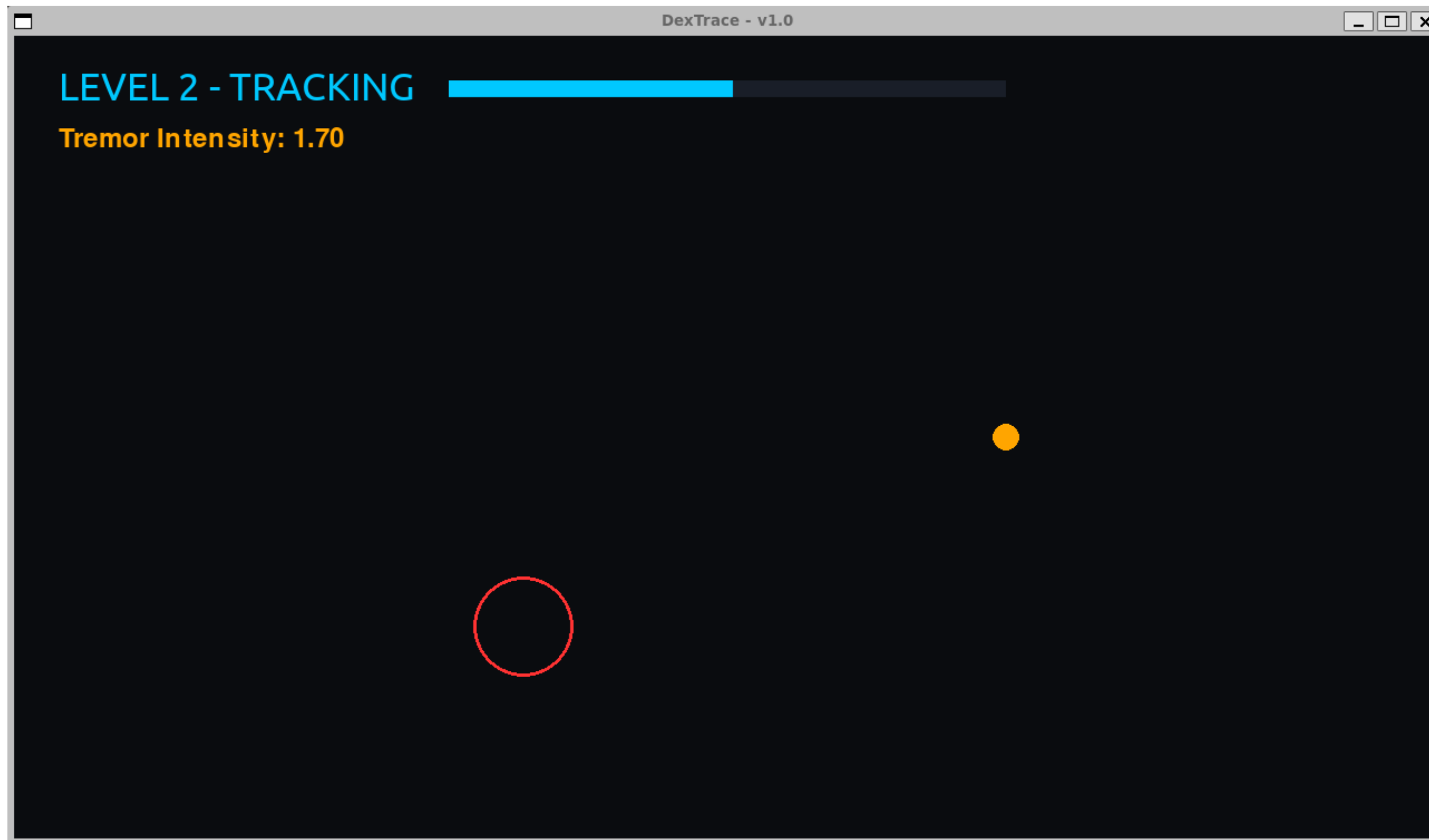
Each level is loaded pushing the button: the rehab_app node (PC) publish the data on the /rehab_config topic

Level 1



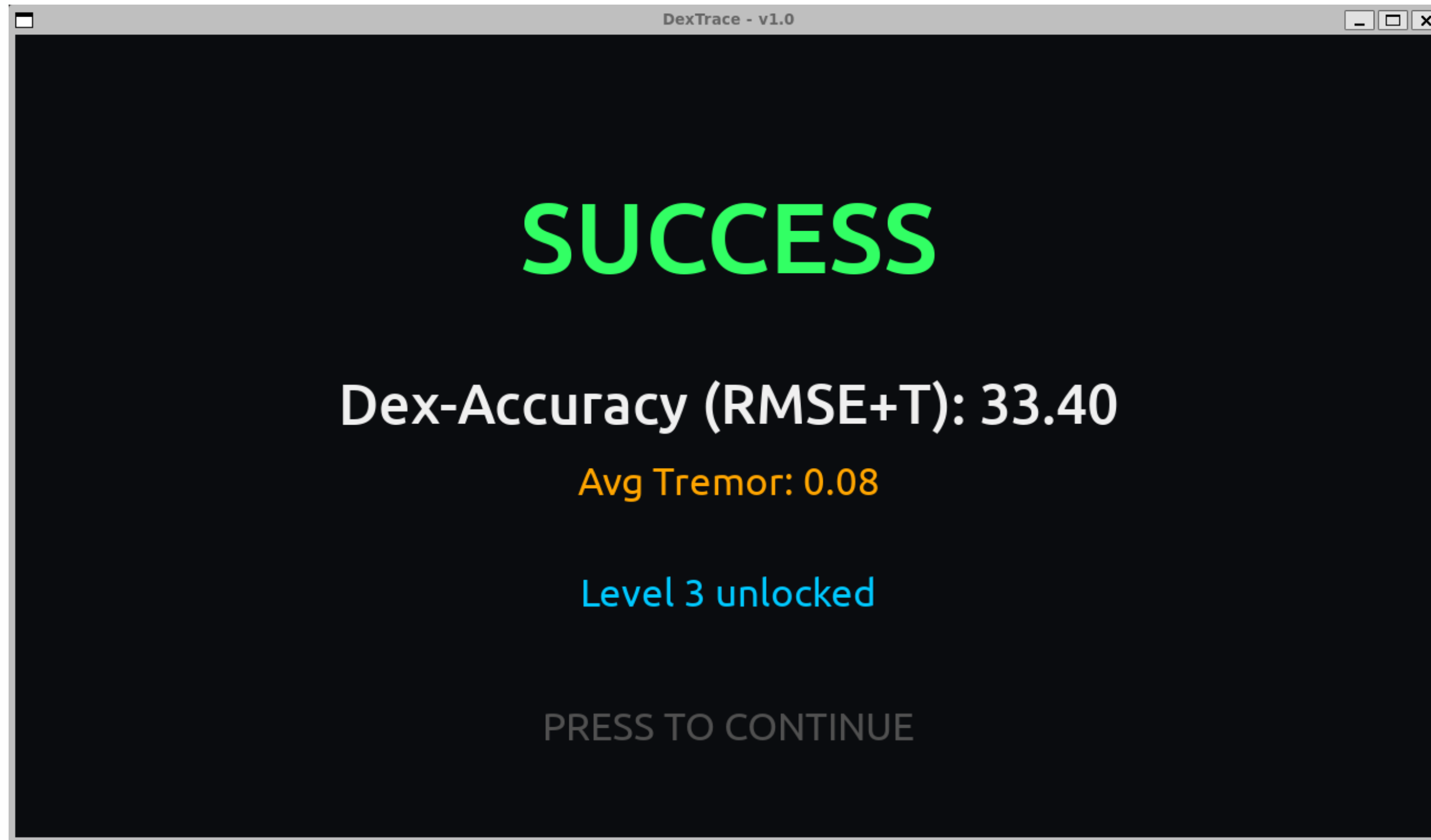
The target color is cursor-position dependant: **red** (not chased), **yellow** (on the border), **green** (chased)

Level 2



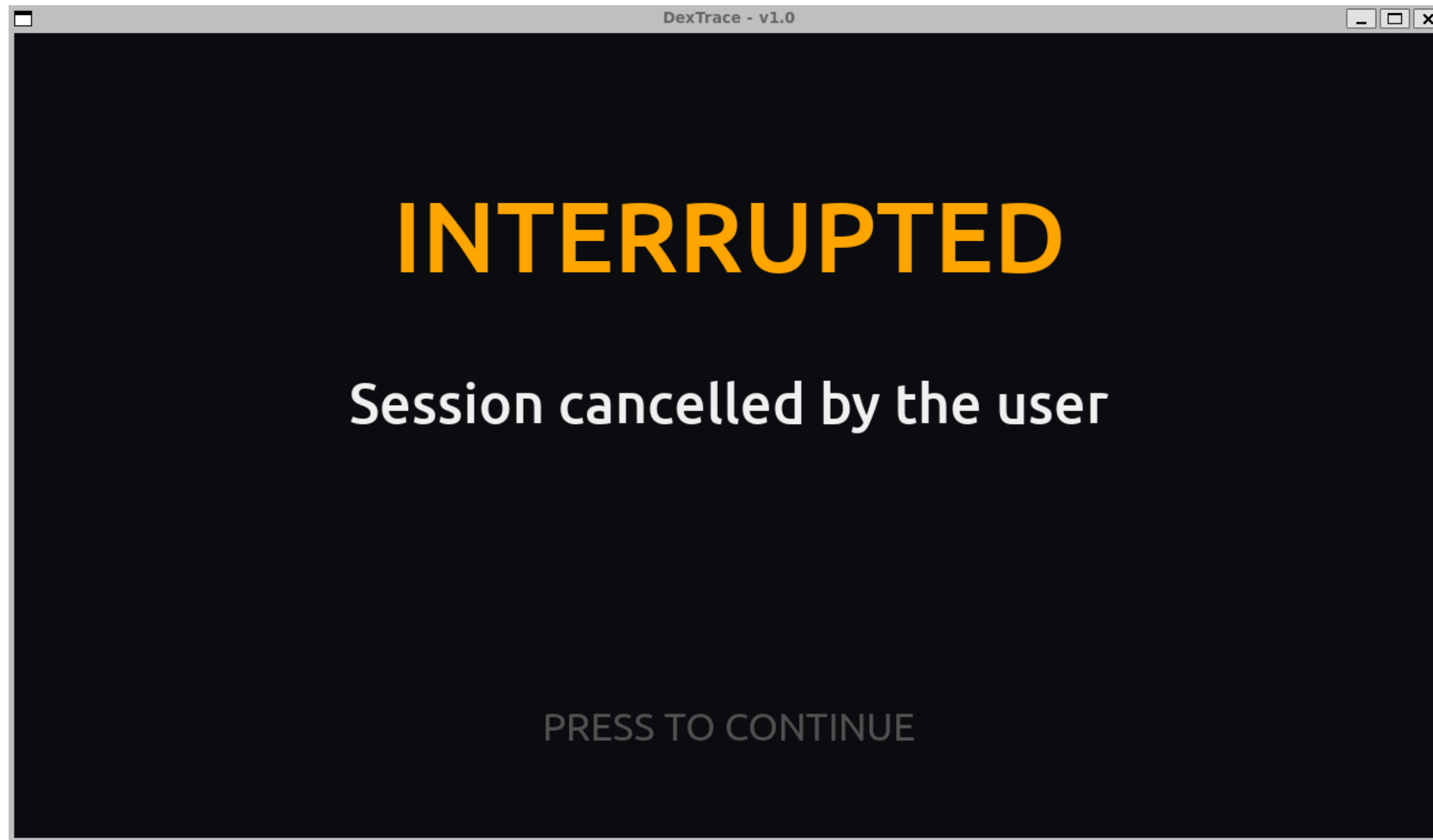
The cursor color became **orange** if the tremor intensity value (upper left) is greater than the threshold

Results



The system shows the evaluation results of each completed (success or fail) exercise

Results - interrupted

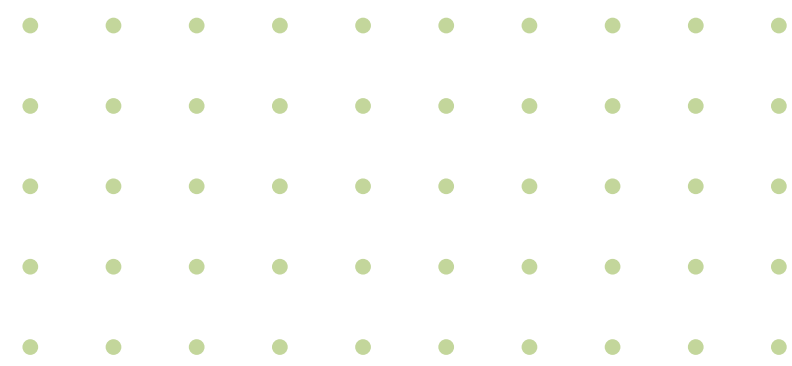


The patient can interrupt the exercise by pressing the button before the timeout expires

Level 3



A tremor intensity value greater than the threshold also introduces a cursor **shake** on the GUI



Thanks for your attention

Developed by
Alex Veronese

Email:
302474@studenti.unimore.it

GitHub:
<https://github.com/alexveronese>

